

DAY - 18: DOM Manipulation with JavaScript - Interactive Guide

 Master DOM Manipulations to Create Dynamic Web Experiences 

Table of Contents

-  DOM Fundamentals
 -  Intro to DOM Manipulation
 -  Creating Elements
 -  Inserting Elements - Before
 -  Inserting Elements - After
 -  Modify Content - innerHTML
 -  Security Risks of innerHTML
 -  Modify Content - textContent
 -  Removing Elements
 -  Traversing the DOM
 -  Manipulating Styles
 -  Manipulating Classes
 -  Working with classList
 -  Controlling Visibility
 -  Project 1: Toggle Visibility
 -  Project 2: Task Manager
 -  Practice Tasks
-

1. DOM Fundamentals

The DOM (Document Object Model) is a programming interface for web documents. It represents the structure of HTML and XML documents as a tree of objects, allowing JavaScript to access and manipulate the content, structure, and styles of a webpage.

Key Concepts:

-  **Document:** The root object representing the entire webpage
-  **Elements:** Objects representing HTML tags (e.g., `<div>` , `<p>` , `<h1>`)

- 🌿 **Nodes:** Everything in the DOM tree (elements, text, comments, etc.)
- 🌲 **Tree Structure:** Parent-child relationship between elements

📁 DOM Tree Visualization:

```
document
├── html
│   ├── head
│   │   ├── title
│   │   │   └── "My Page"
│   │   └── meta
│   └── body
│       ├── h1
│       │   └── "Welcome"
│       ├── p
│       │   └── "This is a paragraph"
│       └── div
│           └── "Content"
```

🔍 Common DOM Selectors:

```
// Get element by ID 🎯
const element = document.getElementById("myId");

// Get elements by class name 🏷️
const elements = document.getElementsByClassName("myClass");

// Get elements by tag name 📄
const paragraphs = document.getElementsByTagName("p");

// Query selector (returns first matching element) 🔍
const firstElement = document.querySelector(".myClass");

// Query selector all (returns all matching elements) 🔍🔍
const allElements = document.querySelectorAll("div.container");
```

💡 **Knowledge Check:** What method would you use to select a specific element with the ID "header"?

▶ 👉 Click to reveal answer

II. 🔄 Intro to DOM Manipulation

DOM manipulation is the process of using JavaScript to modify the structure, content, or style of a webpage after it has loaded. This allows for dynamic and interactive web applications.

🤔 Why DOM Manipulation?

- 🎯 Create dynamic user interfaces
- 💡 Respond to user interactions
- 🔄 Update content without refreshing the page
- ✏️ Create, remove, or modify elements and attributes

🔄 Basic DOM Manipulation Flow:

1. 🔍 **Select** the element(s) you want to manipulate
2. 🛠️ **Modify** the element(s) as needed
3. ✅ **Apply** the changes to update the webpage

💡 **Knowledge Check:** What are two benefits of using DOM manipulation in web development?

▶️ 👉 Click to reveal answer

1. ✨ Creating Elements

JavaScript allows you to create new HTML elements dynamically using the `document.createElement()` method.

🏗️ Creating Elements:

```
// Create a new paragraph element 📄  
const newParagraph = document.createElement("p");  
  
// Create a new div element 🏠  
const newDiv = document.createElement("div");  
  
// Create a new button element ⬤  
const newButton = document.createElement("button");
```

🔧 Adding Content to Created Elements:

```
// Create a new paragraph  
const paragraph = document.createElement("p");  
  
// Add text content ✏️  
paragraph.textContent = "This is a dynamically created paragraph.";  
  
// Set attributes 🏷️  
paragraph.setAttribute("class", "dynamic-content");  
paragraph.id = "dynamic-paragraph";
```

```
// Add to the DOM (we'll cover this more in the next section) 🌳
document.body.appendChild(paragraph);
```

💡 **Knowledge Check:** Write code to create a new `<h2>` element with the text "Hello World" and an ID of "greeting".

▶ 👉 Click to reveal answer

2. ⬅ Inserting Elements - Before

The `insertBefore()` method allows you to insert a new element before a specified existing element.

📄 Syntax:

```
parentElement.insertBefore(newElement, referenceElement);
```

If the `referenceElement` is not declared then it assumes as `null` and then it adds the element to the last

🔍 Example:

```
// Create a new element
const newHeading = document.createElement("h2");
newHeading.textContent = "New Heading";

// Get the reference element
const existingParagraph = document.getElementById("existing-paragraph");

// Get the parent of the reference element
const parentElement = existingParagraph.parentNode;

// Insert the new element before the reference element ➡
parentElement.insertBefore(newHeading, existingParagraph);
```

💡 **Knowledge Check:** Given a div with ID "container" and a paragraph with ID "para1" inside it, how would you insert a new button element before the paragraph?

▶ 👉 Click to reveal answer

3. ➡ Inserting Elements - After

There's no direct `insertAfter()` method in the DOM API, but you can achieve this functionality using a combination of `insertBefore()` and the `nextSibling` property.

Method 1: Using `nextSibling`

```
// To insert elementA after elementB
parentElement.insertBefore(elementA, elementB.nextSibling);
```

Method 2: Using `insertAdjacentElement()`

```
// Insert after the end of the target element
targetElement.insertAdjacentElement("afterend", newElement);
```

Example:

```
// Create a new element
const newParagraph = document.createElement("p");
newParagraph.textContent = "This is inserted after";

// Get the reference element
const referenceElement = document.getElementById("reference");

// Insert after 
referenceElement.insertAdjacentElement("afterend", newParagraph);
```

 **Knowledge Check:** What's the difference between inserting an element before and after another element?

▶  Click to reveal answer

II. Modify Content - `innerHTML`

The `innerHTML` property allows you to get or set the HTML content within an element.

Getting Content:

```
// Get the HTML content of an element
const content = document.getElementById("myElement").innerHTML;
console.log(content); // Shows the HTML inside the element
```

Setting Content:

```
// Set the HTML content of an element
document.getElementById("myElement").innerHTML = "<h3>New Content</h3><p>This replaces everyth
```

💡 **Knowledge Check:** How would you add a bold text and a list to a div with ID "content" using innerHTML?

▶ 🖱️ Click to reveal answer

⚠️ Security Risks of innerHTML

While `innerHTML` is convenient, it poses security risks, particularly when dealing with user input.

💣 Security Concerns:

- 🔒 **Cross-Site Scripting (XSS):** If you insert user-provided content using `innerHTML`, malicious code could be executed
- 🔪 **HTML Injection:** Users could insert HTML that breaks your page layout or functionality

🚫 Example of Vulnerability:

```
// DANGEROUS - Never do this with user input 🦋
const userInput = "<script>alert('Hacked!');</script>";
document.getElementById("output").innerHTML = userInput; // This could execute malicious code
```

✅ Safer Alternatives:

- 📄 Use `textContent` for plain text
- 🔍 Sanitize any HTML content before using `innerHTML`
- 🔧 Use DOM methods like `createElement()` instead

💡 **Knowledge Check:** Why is it dangerous to use innerHTML with unfiltered user input?

▶ 🖱️ Click to reveal answer

📄 Modify Content - textContent

The `textContent` property provides a safer way to modify the text content of an element, as it treats everything as plain text rather than HTML.



Getting Text Content:

```
// Get the text content of an element
const text = document.getElementById("myElement").textContent;
console.log(text); // Shows only the text, without HTML tags
```



Setting Text Content:

```
// Set the text content of an element
document.getElementById("myElement").textContent = "This is safe text content.";
```



Comparison with innerHTML:

```
const element = document.getElementById("demo");

// These have different results:
element.innerHTML = "<b>Bold text</b>"; // ✅ Renders as bold text
element.textContent = "<b>Bold text</b>"; // ⚠️ Renders the literal string with tags visible
```

💡 **Knowledge Check:** What would be displayed if you set `textContent = "<h1>Title</h1>"` on an element?

▶️ 👉 Click to reveal answer



Removing Elements

JavaScript provides multiple ways to remove elements from the DOM.



Method 1: Using remove()

```
// Select the element
const elementToRemove = document.getElementById("unwanted");

// Remove it
elementToRemove.remove();
```



Method 2: Using removeChild()

```
// Select the element to remove
const elementToRemove = document.getElementById("unwanted");
```

```
// Get its parent
const parent = elementToRemove.parentNode;

// Remove the child from the parent
parent.removeChild(elementToRemove);
```

Removing Multiple Elements:

```
// Remove all paragraphs
const paragraphs = document.querySelectorAll("p");
paragraphs.forEach(paragraph => paragraph.remove());
```

Replacing Elements: `replaceChild()` vs `replaceChildren()`

`replaceChild(newChild, oldChild)`

- Replaces **one specific child** with a new one.
- Requires both the **new element** and the **old one to be replaced**.
-  Widely supported across all browsers.

Syntax:

```
parent.replaceChild(newChild, oldChild);
```

Example:

```
const old = document.getElementById("old");
const newElem = document.createElement("h2");
newElem.textContent = "New Heading";

parent.replaceChild(newElem, old);
```

`replaceChildren(...newChildren)`

- Replaces all child elements inside a parent.
- Automatically clears existing children before adding new ones.
- Can insert multiple elements at once.
-  Requires modern browsers (since ~2020).

✓ Syntax:

```
parent.replaceChildren(child1, child2, ...);
```

✓ Example:

```
const h1 = document.createElement("h1");  
h1.textContent = "Heading";  
  
const p = document.createElement("p");  
p.textContent = "Paragraph";  
  
parent.replaceChildren(h1, p);
```

💡 **Knowledge Check:** Write code to remove all elements with the class "temporary" from the page.

▶ 👉 Click to reveal answer

DOM Attribute Manipulation: Read, write and remove attributes

Reading Attributes

```
// Using getAttribute()  
const value = element.getAttribute('attributeName');  
  
// Direct property access  
const value = element.attributeName;  
  
// Using dataset for data-* attributes  
const value = element.dataset.name; // For data-name attribute
```

Writing Attributes

```
// Using setAttribute()  
element.setAttribute('attributeName', 'value');  
  
// Direct property assignment  
element.attributeName = 'value';  
  
// Using dataset for data-* attributes  
element.dataset.name = 'value'; // Sets data-name attribute
```

Removing Attributes

```
// Using removeAttribute()
element.removeAttribute('attributeName');

// Setting to empty string
element.attributeName = '';

// Delete from dataset
delete element.dataset.name;
```

✓ Example

```
const btn = document.getElementById('submitBtn');

// Read
const isDisabled = btn.getAttribute('disabled');
const role = btn.role;

// Write
btn.setAttribute('aria-pressed', 'false');
btn.title = 'Click to submit';

// Remove
btn.removeAttribute('disabled');
```

🧭 Traversing the DOM

DOM traversal allows you to navigate through the DOM tree structure, moving from one element to related elements.

🧭 Common Traversal Properties:

Property	Description
parentNode	⬆ Returns the parent node of an element
childNodes	⬇ Returns all child nodes as a NodeList
children	👤 Returns all child elements (excluding text and comment nodes)
firstChild	1 Returns the first child node
firstElementChild	1 Returns the first child element
lastChild	⬅ Returns the last child node

Property	Description
<code>lastElementChild</code>	 Returns the last child element
<code>nextSibling</code>	 Returns the next sibling node
<code>nextElementSibling</code>	 Returns the next sibling element
<code>previousSibling</code>	 Returns the previous sibling node
<code>previousElementSibling</code>	 Returns the previous sibling element

Examples:

```
// Get the parent of an element 
const parent = document.getElementById("child").parentNode;

// Get all children of an element 
const children = document.getElementById("parent").children;

// Get the next sibling element 
const nextElement = document.getElementById("current").nextElementSibling;

// Get the first child element 
const firstChild = document.getElementById("parent").firstElementChild;
```

 **Knowledge Check:** Given a paragraph element, how would you select its parent's last child?

▶  Click to reveal answer

Manipulating Styles

JavaScript can manipulate an element's CSS styles directly through the `style` property.

Setting Individual Styles:

```
const element = document.getElementById("myElement");

// Set individual style properties
element.style.color = "blue";
element.style.backgroundColor = "yellow";
element.style.fontSize = "18px";
element.style.padding = "10px";
element.style.border = "1px solid black";
```

Note on Style Property Names:

CSS properties with hyphens are converted to camelCase in JavaScript:

- `background-color` → `backgroundColor`
- `font-size` → `fontSize`
- `border-radius` → `borderRadius`

Setting Multiple Styles at Once:

```
const element = document.getElementById("myElement");

// Using Object.assign to set multiple styles
Object.assign(element.style, {
  color: "white",
  backgroundColor: "black",
  padding: "20px",
  borderRadius: "5px"
});
```

Getting Computed Styles:

```
// Get all computed styles of an element
const computedStyles = window.getComputedStyle(element);
console.log(computedStyles.color); // RGB value of the color
```

 **Knowledge Check:** How would you change the background color, text color, and font size of a button with ID "submit-btn"?

▶  Click to reveal answer

Manipulating Classes

Classes are a powerful way to apply styles to elements. JavaScript provides several methods to manipulate classes.

Adding Classes:

```
const element = document.getElementById("myElement");

// Add a class (old way)
element.className += " new-class";
```

```
// Better way: use classList.add() ✨  
element.classList.add("new-class");
```

— Removing Classes:

```
const element = document.getElementById("myElement");  
  
// Remove a class using classList  
element.classList.remove("old-class");
```

🔄 Setting/Replacing All Classes:

```
const element = document.getElementById("myElement");  
  
// Replace all classes with a new set  
element.className = "class1 class2 class3";
```

💡 **Knowledge Check:** What is the difference between using `className` and `classList` to manipulate classes?

▶ 👉 Click to reveal answer

🔄 Working with classList

The `classList` property provides methods to work with classes more effectively than direct manipulation of the `className` property.

🔧 Key Methods:

```
const element = document.getElementById("myElement");  
  
// Add one or more classes +  
element.classList.add("class1", "class2");  
  
// Remove one or more classes -  
element.classList.remove("class1", "class2");  
  
// Toggle a class (add if absent, remove if present) 🔄  
element.classList.toggle("active");  
  
// Check if an element has a specific class ✅  
const hasClass = element.classList.contains("important");
```

```
// Replace one class with another 🔄  
element.classList.replace("old-class", "new-class");
```

🌙 Example: Toggle Dark Mode

```
const toggleButton = document.getElementById("dark-mode-toggle");  
const body = document.body;  
  
toggleButton.addEventListener("click", function() {  
  body.classList.toggle("dark-mode");  
  
  if (body.classList.contains("dark-mode")) {  
    toggleButton.textContent = "Switch to Light Mode ☀️";  
  } else {  
    toggleButton.textContent = "Switch to Dark Mode 🌙";  
  }  
});
```

💡 **Knowledge Check:** Write code to toggle a "highlighted" class on a paragraph when it's clicked.

▶️ 🖱️ Click to reveal answer

👁️ Controlling Visibility

JavaScript provides several methods to control the visibility of elements.

🔍 Method 1: Using display property

```
// Hide an element 🙈  
element.style.display = "none";  
  
// Show an element 🙌  
element.style.display = "block"; // or "inline", "flex", etc.
```

🔍 Method 2: Using visibility property

```
// Hide an element (space is still occupied) 🙈  
element.style.visibility = "hidden";  
  
// Show an element  
element.style.visibility = "visible";
```

Method 3: Using opacity

```
// Hide an element (with transition possible)   
element.style.opacity = "0";  
  
// Show an element  
element.style.opacity = "1";
```

Method 4: Using classList

```
// Using a CSS class with display: none  
element.classList.add("hidden");  
  
// Remove the class to show again  
element.classList.remove("hidden");
```

 **Knowledge Check:** What's the difference between setting `display: none` and `visibility: hidden` ?

▶  Click to reveal answer

Project 1 - Toggle Visibility

Let's create a simple project to apply what we've learned: a button that toggles the visibility of a text element.

HTML Structure:

```
<div class="container">  
  <h2>Toggle Visibility Project</h2>  
  <button id="toggle-button">Toggle Text</button>  
  <p id="toggle-text">This text will be toggled!</p>  
</div>
```

CSS:

```
.container {  
  text-align: center;  
  margin-top: 50px;  
}  
  
button {  
  padding: 10px 20px;
```

```
background-color: #4CAF50;
color: white;
border: none;
border-radius: 4px;
cursor: pointer;
}

.hidden {
  display: none;
}
```

JavaScript:

```
// Get references to the elements
const toggleButton = document.getElementById("toggle-button");
const toggleText = document.getElementById("toggle-text");

// Add event listener to the button
toggleButton.addEventListener("click", function() {
  // Toggle the visibility of the text
  toggleText.classList.toggle("hidden");

  // Update button text based on visibility
  if (toggleText.classList.contains("hidden")) {
    toggleButton.textContent = "Show Text 👁";
  } else {
    toggleButton.textContent = "Hide Text 🙈";
  }
});
```

This project demonstrates:

-  Selecting elements with `getElementById`
-  Adding event listeners
-  Toggle classes with `classList.toggle`
-  Conditional logic with `classList.contains`
-  Changing text content

 **Knowledge Check:** How would you modify this project to fade the text in and out instead of hiding it completely?

Project 2 - Task Manager

Let's create a more complex project: a simple task manager that allows adding, marking as complete, and deleting tasks.

HTML Structure:

```
<div class="task-manager">
  <h2>Task Manager</h2>

  <div class="add-task">
    <input type="text" id="task-input" placeholder="Enter a new task...">
    <button id="add-button">Add Task</button>
  </div>

  <ul id="task-list">
    <!-- Tasks will be added here dynamically -->
  </ul>
</div>
```

CSS:

```
.task-manager {
  max-width: 500px;
  margin: 0 auto;
  padding: 20px;
  font-family: Arial, sans-serif;
}

.add-task {
  display: flex;
  margin-bottom: 20px;
}

#task-input {
  flex-grow: 1;
  padding: 8px;
  border: 1px solid #ddd;
  border-radius: 4px 0 0 4px;
}

#add-button {
  padding: 8px 15px;
  background-color: #4CAF50;
  color: white;
  border: none;
  border-radius: 0 4px 4px 0;
  cursor: pointer;
}

#task-list {
  list-style-type: none;
  padding: 0;
}
```

```

.task-item {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 10px;
  margin-bottom: 10px;
  background-color: #f9f9f9;
  border-radius: 4px;
}

.completed {
  text-decoration: line-through;
  color: #888;
  background-color: #f0f0f0;
}

.task-actions button {
  margin-left: 5px;
  padding: 5px 10px;
  border: none;
  border-radius: 3px;
  cursor: pointer;
}

.complete-btn {
  background-color: #2196F3;
  color: white;
}

.delete-btn {
  background-color: #f44336;
  color: white;
}

```

JavaScript:

```

// Get references to DOM elements
const taskInput = document.getElementById("task-input");
const addButton = document.getElementById("add-button");
const taskList = document.getElementById("task-list");

// Function to create a new task
function createTask(taskText) {
  // Create list item
  const taskItem = document.createElement("li");
  taskItem.className = "task-item";

  // Create task content
  const taskContent = document.createElement("span");
  taskContent.className = "task-content";
  taskContent.textContent = taskText;
}

```

```

// Create action buttons container
const taskActions = document.createElement("div");
taskActions.className = "task-actions";

// Create complete button
const completeButton = document.createElement("button");
completeButton.className = "complete-btn";
completeButton.textContent = "Complete";
completeButton.addEventListener("click", function() {
  taskItem.classList.toggle("completed");
  completeButton.textContent = taskItem.classList.contains("completed")
    ? "Undo ↺ "
    : "Complete ✓";
});

// Create delete button
const deleteButton = document.createElement("button");
deleteButton.className = "delete-btn";
deleteButton.textContent = "Delete";
deleteButton.addEventListener("click", function() {
  taskItem.remove();
});

// Assemble the task item
taskActions.appendChild(completeButton);
taskActions.appendChild(deleteButton);
taskItem.appendChild(taskContent);
taskItem.appendChild(taskActions);

// Add the task to the list
taskList.appendChild(taskItem);
}

// Event listener for add button
addButton.addEventListener("click", function() {
  const taskText = taskInput.value.trim();

  if (taskText !== "") {
    createTask(taskText);
    taskInput.value = ""; // Clear the input
    taskInput.focus(); // Focus back on input
  }
});

// Event listener for Enter key in input
taskInput.addEventListener("keypress", function(event) {
  if (event.key === "Enter") {
    addButton.click(); // Trigger the add button click
  }
});

```

This project demonstrates:

- ✨ Creating multiple elements with `createElement`
- 💡 Adding event listeners to dynamically created elements
- 🔗 Appending child elements
- 🔄 Toggling classes
- 🗑️ Removing elements
- 📄 Working with form inputs
- 🗨️ Event delegation
- 🔍 DOM traversal

💡 **Knowledge Check:** How would you enhance this task manager to save tasks to local storage so they persist when the page is refreshed?

💪 Practice Tasks

🔍 Task 1: Element Selection

Write JavaScript code to:

1. Select all paragraphs on a page
2. Select the first element with class "highlight"
3. Select all elements with both "box" and "active" classes
4. Select all direct children of a div with ID "container"

✨ Task 2: Create & Modify

Write code to:

1. Create a new button element with text "Click Me" and append it to a div with ID "buttons"
2. Create an image element with src "image.jpg" and alt text "My Image"
3. Create a link element that opens "<https://example.com>" in a new tab

🔙 Task 3: Element Placement

Given this HTML:

```
<div id="container">
  <p id="first">First paragraph</p>
  <p id="second">Second paragraph</p>
</div>
```

Write JavaScript to:

1. Insert a new paragraph between the first and second paragraphs
2. Add a heading before the first paragraph
3. Add a button after the second paragraph

Task 4: Content Manipulation

Write code to:

1. Replace the text of a paragraph with ID "info" to "Updated information"
2. Add bold text to an existing paragraph without removing its current content
3. Safely set the content of a div to include user input (without XSS risk)

Task 5: Traversal Challenge

Given a deeply nested structure, write code to:

1. Navigate from a button with ID "deep-button" to its grandparent element
2. Get the next sibling of the parent of an element with ID "nested"
3. Find all child elements that are paragraphs in a section with ID "content"

Task 6: Style Manipulation

Write code to:

1. Make all paragraphs with class "highlight" have a yellow background and bold text
2. Double the font size of an element when it's clicked
3. Create a function that toggles between light and dark themes by changing multiple style properties

Task 7: Class Manipulation

Write code to:

1. Add "active" class to a button when clicked and remove it from all other buttons
2. Toggle between three different appearance classes on an element
3. Check if an element has a specific class and act accordingly

Task 8: Visibility Toggle

Create a small program that:

1. Has three buttons labeled "Hide", "Show", and "Toggle"
2. Each button performs its action on a target div

3. The "Toggle" button should also change its text between "Hide" and "Show"

Task 9: Mini-Project: Tabbed Interface

Create a simple tabbed interface with:

1. 3 tabs that show different content when clicked
2. Only one content panel visible at a time
3. Active tab highlighted with a different style

Task 10: DOM Manipulation Challenge

Build a "Color Palette Generator" that:

1. Creates 5 div elements in a row
2. Each div should have a random background color
3. When clicked, a div should change to a new random color
4. Add a "Reset" button that regenerates all colors

Let's Connect & Level Up Together

Follow me for daily JavaScript tips, insightful notes, and project-based learning.

 [X \(Twitter\)](#)

 [LinkedIn](#)

 Dive into the full notes on GitHub → [40 Days JavaScript](#)

© 2025 • Crafted with  by **Neeraj**

 *Next: Day - 19: Master JavaScript Events* 

 Happy JavaScript coding! 

 **Happy DOM Manipulation!** 

Master these concepts to create dynamic, interactive web experiences.